

Distribution Management System

Md Abir Hossain

July 2026

Abstract

This report presents the design and implementation of a **Distribution Management System (DMS)**, a console-based application written in the C programming language. The system models a real-life product distribution chain consisting of three levels: a **company**, its **dealers**, and the **retailers** connected to those dealers. The company holds products in a central warehouse, dealers purchase stock from the company, and retailers purchase stock from their assigned dealer. The project was intentionally built using only fundamental programming concepts, namely variables, arrays, loops, conditional statements and functions, so that every part of the code remains easy to read and explain. The complete program is divided into six source modules that communicate through header files, demonstrating how a beginner-friendly project can still follow a clean, modular structure.

Keywords: C programming, distribution chain, modular design, arrays, stock management, console application

1. Introduction

1.1 Background

In the real world, most products do not travel directly from a manufacturer to a customer. Instead they pass through a distribution chain. A company produces goods and stores them in a warehouse. Dealers (also called distributors) buy these goods in bulk and store them regionally. Retailers, which are the local shops that customers actually visit, buy smaller quantities from a nearby dealer. Keeping track of how much stock exists at every level of this chain is a classic inventory management problem.

This project simulates that exact flow in software. Although commercial distribution systems are large database-driven applications, the same core logic, checking stock, transferring quantities and recording the result, can be expressed with simple arrays and functions in C. Building it this way makes the underlying idea very clear, because nothing is hidden behind frameworks or libraries.

1.2 Motivation

The main motivation behind this project was to practise writing a **multi-file C program** that solves a realistic problem while staying inside the topics covered in a first programming course. Most beginner exercises fit in a single file, but real software is always split into modules. This project was a chance to learn how separate .c files share data and functions through header files, and how a program can be organised so that each file has one clear responsibility.

1.3 Objectives

- To build a working simulation of a three-level distribution chain (company, dealer, retailer).
- To allow each type of user to view the stock that belongs to them.
- To let dealers order products from the company, and retailers order from their dealer, with automatic stock transfer.
- To reject invalid operations such as ordering more than the available stock or entering an unknown product ID.
- To keep the entire codebase limited to basic C concepts: if-else, loops, arrays and functions.
- To organise the program into separate modules that communicate through header files.

1.4 Scope of the Project

The system is a console application that runs in a terminal. All data lives in memory while the program runs; when the program exits, the state resets to the default sample data. Features such as file storage, passwords, prices and payments were deliberately left out to keep the focus on the distribution logic itself. These are discussed later as future improvements.

2. System Overview

2.1 The Distribution Chain

The whole system is built around one idea: **stock flows downward through the chain, and orders flow upward**. A retailer asks its dealer for products, and the dealer asks the company. Every successful order subtracts a quantity from the seller's stock and adds the same quantity to the buyer's stock, so the total amount of product in the system never changes on its own.

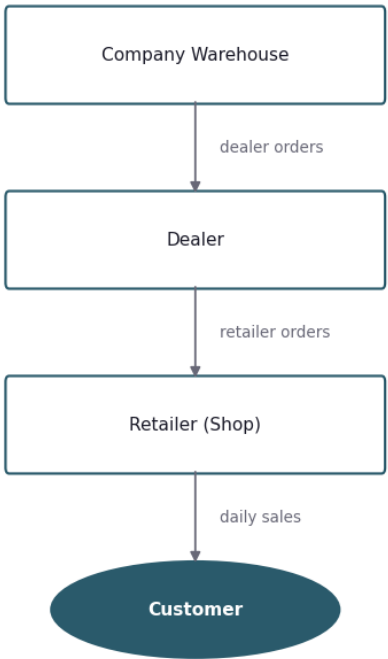


Figure 1: Product flow through the three-level distribution chain

2.2 The Three User Roles

Role	Can View	Can Do
Company	All dealers, all products, warehouse stock	Monitors the whole chain

Dealer	Own stock, company product list	Order products from the company
Retailer	Own stock, assigned dealer's product list	Order products from its dealer

Capabilities of each user role

Each retailer is permanently linked to exactly one dealer. For example, in the default data, Mirpur Shop buys only from Dhaka Dealer, and Gulshan Shop buys only from Chittagong Dealer. This mirrors how territorial distribution works in practice: a shop has an assigned local distributor.

2.3 Default Sample Data

So that the program can be tested immediately, it starts with a small set of sample data loaded by the `init_data()` function. Three products exist in the system, and every dealer and retailer begins with some stock already on hand.

Product	Company	Dhaka Dealer	Chittagong Dealer	Mirpur Shop	Gulshan Shop
Rice	100	20	25	5	4
Oil	80	15	12	3	6
Sugar	60	10	18	2	1

Initial stock (in units) at every level of the chain

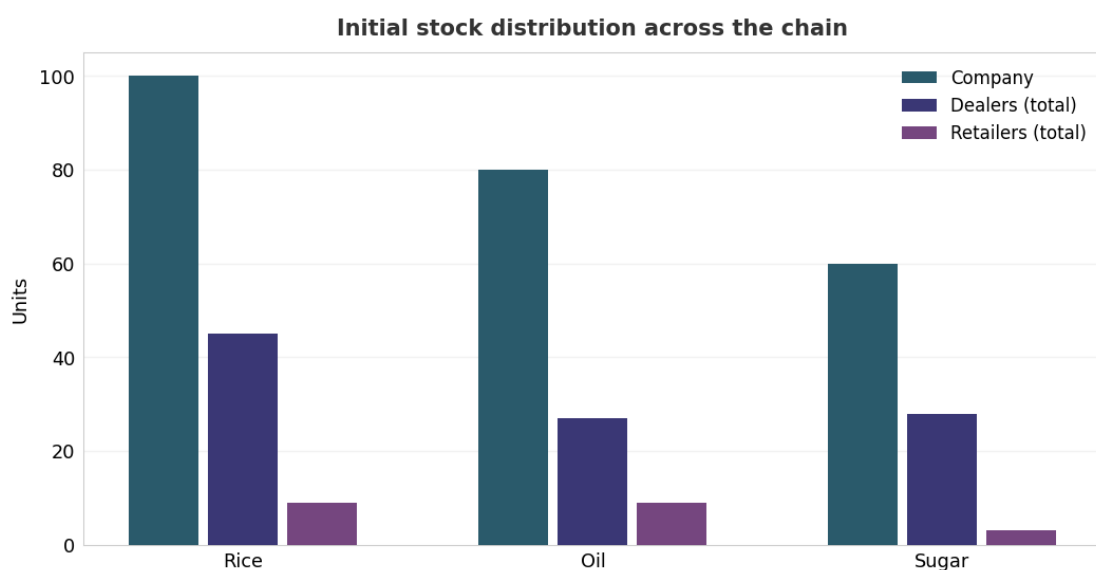


Figure 2: Most stock starts at the company and thins out toward the retailers, just like a real supply chain.

3. System Design

3.1 Modular Architecture

The program is split into six modules. Each module is a pair of files: a **header file (.h)** that declares what the module offers, and a **source file (.c)** that contains the actual code. Any module that wants to use another module simply includes its header. This is the standard way C programs are organised, and it means each file can be read and understood on its own.

Module	Files	Responsibility
Main	main.c	Program entry point; shows the main menu and routes to the right panel
Data	data.h / data.c	Declares and stores all shared arrays; loads the default sample data
Display	display.h / display.c	All printing helpers: menus, tables, error and success messages
Company	company.h / company.c	Company panel: view dealers, products and warehouse stock
Dealer	dealer.h / dealer.c	Dealer panel: login, view stock, order from the company
Retailer	retailer.h / retailer.c	Retailer panel: login, view stock, order from the dealer

The six modules and what each one is responsible for

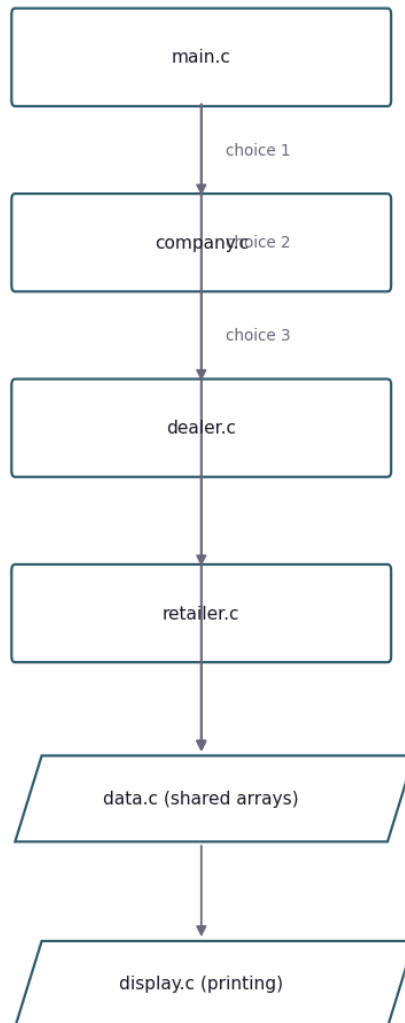


Figure 3: Module relationships: the three panels all read and write the shared data module and print through the display module

3.2 How the Modules Communicate

All shared information lives in **data.c** as global arrays. The header file **data.h** re-declares each array with the **extern** keyword, which tells the compiler that the variable exists in another file. When **dealer.c** includes **data.h**, it gains access to the exact same arrays that **company.c** and **retailer.c** use. This is why an order placed in the dealer panel is immediately visible in the company panel: there is only one copy of the data, shared by everyone.

```

/* data.h (declaration - visible to every module) */
extern int  company_stock[MAX_PRODUCTS];
extern int  dealer_stock[MAX_DEALERS][MAX_PRODUCTS];

/* data.c (definition - the single real copy) */
int company_stock[MAX_PRODUCTS];
int dealer_stock[MAX_DEALERS][MAX_PRODUCTS];

```

Key idea: a variable is DEFINED once in one .c file, but can be DECLARED with extern in a header so that every other file can use it.

3.3 Data Structures

Instead of structures or dynamic memory, the project uses **parallel arrays**: several arrays that share the same index. For example, index 0 of `product_id`, `product_name` and `company_stock` together describe the first product (ID 1, "Rice", 100 units). Stock owned by dealers and retailers is stored in **two-dimensional arrays**, where the row selects the dealer or retailer and the column selects the product.

Array	Type	Meaning
<code>product_id[10]</code>	int	Numeric ID of each product
<code>product_name[10][50]</code>	char	Name of each product (a string per product)
<code>company_stock[10]</code>	int	Units of each product in the company warehouse
<code>dealer_id[5], dealer_name[5][50]</code>	int / char	ID and name of each dealer
<code>dealer_stock[5][10]</code>	int	<code>dealer_stock[d][p]</code> = units of product p held by dealer d
<code>retailer_id[5], retailer_name[5][50]</code>	int / char	ID and name of each retailer
<code>retailer_dealer[5]</code>	int	ID of the dealer this retailer is linked to
<code>retailer_stock[5][10]</code>	int	<code>retailer_stock[r][p]</code> = units of product p held by retailer r

The complete data model of the system

The sizes 10 and 5 come from the constants `MAX_PRODUCTS`, `MAX_DEALERS` and `MAX_RETAILERS` defined in `data.h`, so the capacity of the whole system can be changed

by editing a single line.

4. Implementation

4.1 Programming Concepts Used

Concept	Where It Is Used
if / else if	Menu routing, input validation, stock checking
do-while loop	Every menu repeats until the user chooses 0 to exit
for loop	Printing lists and tables; searching arrays by ID
1-D arrays	Product IDs, names and company stock
2-D arrays	Dealer and retailer stock (entity x product)
Functions	Each feature is one small function with a clear name
Multiple source files	Six modules linked together into one executable
extern variables	Sharing the data arrays between all modules

Every language feature in the project is a fundamental one

4.2 The Main Menu (main.c)

Execution starts in `main()`. It first calls `init_data()` to load the sample data, then enters a do-while loop that keeps showing the main menu until the user types 0. A chain of if-else statements sends the user to the correct panel.

c

```
int main() {
    int choice;

    init_data();

    do {
        print_menu_header("DISTRIBUTION MANAGEMENT SYSTEM");
        print_menu_item(1, "Company Login");
        print_menu_item(2, "Dealer Login");
        print_menu_item(3, "Retailer Login");
        print_menu_item(0, "Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        if (choice == 1)        company_menu();
        else if (choice == 2) dealer_menu();
        else if (choice == 3) retailer_menu();
        else if (choice != 0)
            print_error("Invalid choice. Please try again.");
    } while (choice != 0);

    return 0;
}
```

4.3 Searching by ID (the found flag pattern)

Users refer to products, dealers and retailers by their ID numbers, but arrays are accessed by index. The program converts an ID to an index with a simple linear search. A flag variable named **found** starts at 0 and becomes 1 if the loop finds a match; if the loop finishes with found still 0, the ID does not exist and an error is shown. This little pattern appears everywhere in the project.

C

```
found = 0;
product_index = 0;
for (i = 0; i < product_total; i++) {
    if (product_id[i] == product_id_input) {
        found = 1;
        product_index = i;
        break;
    }
}

if (found == 0) {
    print_error("Product not found. Check the table above.");
    return;
}
```

4.4 Placing an Order

Ordering is the heart of the system. Before any stock moves, the program runs three checks in order: the product must exist, the quantity must be positive, and the seller must actually have that many units. Only when all three checks pass does the transfer happen, and it is just two lines of arithmetic: subtract from the seller, add to the buyer.

C

```
/* from order_from_company() in dealer.c */
if (company_stock[product_index] < quantity) {
    print_error("Not enough stock in company warehouse.");
    return;
}

company_stock[product_index] -= quantity;
dealer_stock[dealer_index][product_index] += quantity;

print_order_summary(product_name[product_index], quantity,
                    "Company", dealer_name[dealer_index]);
```

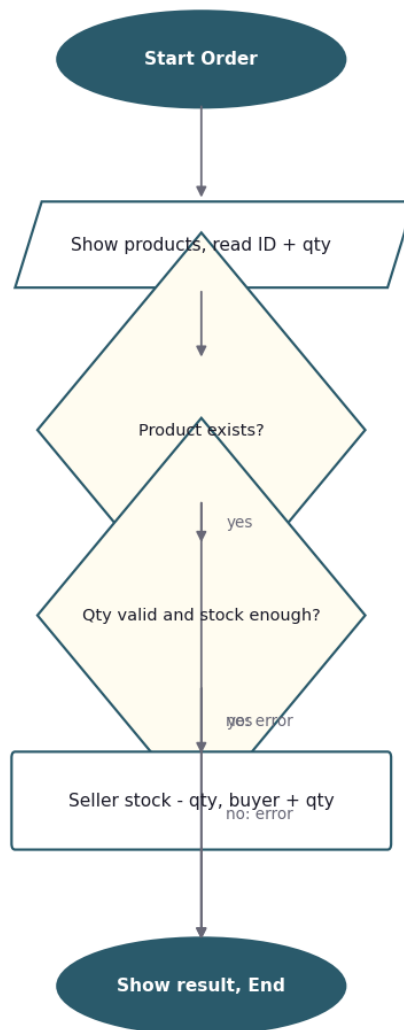


Figure 4: Order processing: three validation gates protect the stock transfer

4.5 The Display Module

All printing goes through helper functions in `display.c`, such as `print_menu_header()`, `print_table_product_row()` and `print_order_summary()`. Centralising the printing has two benefits: every screen in the program looks consistent, and the panel files stay short because they never repeat formatting code. The tables are drawn with plain text characters using `printf` width specifiers such as `%-20s`, which pads a string to 20 characters so the columns line up.

c

```
void print_table_product_header() {
    printf("+-----+-----+-----+-----+\\n");
    printf("| ID   | Product Name       | Stock   |\\n");
    printf("+-----+-----+-----+-----+\\n");
}

void print_table_product_row(int id, char name[], int stock) {
    printf("| %-4d | %-20s | %-10d |\\n", id, name, stock);
}
```

5. Compilation and Running

Because the program is spread across six .c files, all of them must be compiled and linked together. The GCC compiler does this in one command. Each .c file is compiled separately into machine code, and the linker then joins them into a single executable named dms. If any module called a function that no other module defines, the linker would report an undefined reference error, which is how C enforces that the modules fit together.

BASH

```
cd DMS
gcc -o dms main.c data.c company.c dealer.c retailer.c display.c
./dms
```

5.1 Sample Session

The following is a real capture of the program running. The user logs in as Dhaka Dealer and orders 10 units of Rice from the company. Notice how the product table is shown before the order so the user can see IDs and available stock.

```
=====
DEALER PANEL
=====

Welcome, Dhaka Dealer

[1] View My Stock
[2] View Company Products
[3] Order From Company
[0] Logout

Enter your choice: 3

--- Company Products (Available to Order) ---

+-----+-----+-----+
| ID  | Product Name      | Stock  |
+-----+-----+-----+
| 1   | Rice              | 100    |
| 2   | Oil               | 80     |
| 3   | Sugar            | 60     |
+-----+-----+-----+

Enter Product ID: 1
Enter Quantity: 10

=====
ORDER PLACED SUCCESSFULLY
=====

Product   : Rice
Quantity  : 10 units
From      : Company
To        : Dhaka Dealer
=====
```

After this order, the company's Rice stock drops from 100 to 90 and Dhaka Dealer's Rice stock rises from 20 to 30. Both changes can be verified immediately from the company panel and the dealer panel, because both panels read the same shared arrays.

6. Testing

The system was tested manually by walking through every menu path and deliberately entering both valid and invalid input. The table below lists the main test cases and the observed behaviour.

#	Test Case	Input	Expected Result	Status
1	Company views products	Main menu 1, then 2	Table of 3 products with stock	Pass
2	Dealer login with valid ID	Dealer ID 1	Login success, panel opens	Pass
3	Dealer login with invalid ID	Dealer ID 9	Error: dealer not found	Pass
4	Valid order	Rice, qty 10	Stock moves company to dealer	Pass
5	Order more than stock	Rice, qty 5000	Error: not enough stock	Pass
6	Order zero or negative qty	qty -5	Error: quantity must be positive	Pass
7	Order unknown product	Product ID 99	Error: product not found	Pass
8	Retailer orders from dealer	Mirpur Shop, Oil, qty 2	Stock moves dealer to retailer	Pass
9	Stock consistency	Order then re-check both panels	Totals stay balanced	Pass
10	Invalid menu choice	Choice 7	Error, menu shown again	Pass

Manual test cases and results

The most important invariant tested: after any order, the amount removed from the seller always equals the amount added to the buyer, so no stock is ever created or destroyed by mistake.

7. Limitations and Future Work

7.1 Current Limitations

- Data is not saved to disk, so all changes are lost when the program closes.
- There are no prices or payments; orders only move quantities.
- Products, dealers and retailers cannot be added while the program is running; they are fixed in `init_data()`.
- There is no password protection; anyone can log in as any dealer or retailer by choosing an ID.
- Non-numeric input to `scanf` can confuse the menus, since input is read as plain integers.

7.2 Future Improvements

- Save and load the stock data using C file handling (`fopen`, `fprintf`, `fscanf`), so the state survives between runs.
- Add product prices and calculate the total cost of each order.
- Add functions to register new products, dealers and retailers at runtime.
- Keep a simple order history log so past transactions can be reviewed.
- Add low-stock warnings when a product falls below a threshold.

8. Conclusion

The Distribution Management System achieves everything it set out to do. It models a realistic three-level supply chain, enforces sensible rules on every stock transfer, and presents its information in clean, readable console tables. More importantly, it proves that a well-structured, multi-file program does not require advanced language features: the entire system is built from arrays, loops, conditionals and functions. Working on this project gave practical experience in dividing a problem into modules, sharing data between source files with extern declarations, validating user input, and designing text output that people can actually read. The final program compiles cleanly with GCC and behaved correctly in all ten test scenarios.

References

- [1] Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- [2] GNU Project. (2024). GCC, the GNU Compiler Collection - Online Documentation.
<https://gcc.gnu.org/onlinedocs/>
- [3] TutorialsPoint. (2024). C Programming Tutorial - Multi-file Programs and Header Files.
<https://www.tutorialspoint.com/cprogramming/>
- [4] Chopra, S., & Meindl, P. (2019). Supply Chain Management: Strategy, Planning, and Operation (7th ed.). Pearson.